MailVault Architecture and Trade-off Analysis

AWS-managed reference architecture vs. open-source/hybrid architecture
Prepared from the MailVault PRD and the proposed technology stacks.

Primary objective	Decision needed	Recommended path
Convert scanned mail into searchable, actionable digital records	Choose between higher managed-service velocity and lower-cost open-source control	AWS-managed MVP, with model/provider abstraction to preserve future portability

Scope

This document turns the PRD into two implementation options: (1) an AWS-heavy managed architecture and (2) an open-source/hybrid architecture using open components plus external LLM APIs.

It also proposes the document-scanning workflow for the website, the ingestion pipeline, and the document/email classification approach for the categories bills, bank, insurance, medical, government, personal, and legal.

The PRD requires scan, OCR, storage, categorization, AI summary, question answering, search, and reminders. The recommendation below keeps those capabilities intact while comparing cost, efficiency, performance, and operational complexity.

Decision summary

For an MVP, the strongest path is a managed AWS architecture for ingestion, OCR, storage, and orchestration, because it minimizes platform engineering overhead and reduces the number of moving parts.

For long-term cost control and model portability, keep the AI and classification layer behind a service boundary so the application can switch among Bedrock, OpenAI, Anthropic, or self-hosted models without changing the product flows.

Do not rely on a generic 'Google' classifier by default. The right method here is a hybrid document classifier: document-layout signals + OCR text + rules + LLM verification, with confidence thresholds and human-review fallback. Google Document AI has custom classifier and splitter products, but this use case does not require Google specifically if AWS or open-source paths are already preferred.

1. PRD requirements translated into architecture needs

PRD capability	Architecture implication	Priority
Mail scanning	Need mobile/web camera capture, edge detection, crop, enhancement, upload, and retry path for blurry images.	High
OCR extraction	Need text + structure extraction from bills, forms, letters, and multi-page packets.	High
Storage	Need immutable original files, derived text, metadata, summaries, embeddings, and audit history.	High
Categorization	Need taxonomy classifier that works even when OCR is noisy and layouts vary by sender.	High
AI summary + Q&A	Need retrieval over extracted text plus prompt templates for due dates, payments, actions, and importance.	High
Search + reminders	Need metadata index, date/entity extraction, and workflow/event engine for reminders.	Medium
Security	Need per-user isolation, encryption, strict access control, and production auditability.	High

2. AWS-managed reference architecture

Recommended services

- Frontend/Web: website or web app with browser camera capture, document preview, crop confirmation, and authenticated upload.
- Storage: Amazon S3 for originals, processed images, and derived artifacts.
- OCR: Amazon Textract for text, forms, tables, and document analysis.
- Metadata + operational data: DynamoDB for mail records, status, due dates, category, and workflow state.
- Retrieval and AI: Amazon Bedrock Knowledge Bases for RAG over extracted content; Bedrock model endpoints for summarization and Q&A.
- Agent/runtime layer: Bedrock AgentCore or an application-owned orchestration service for agentic flows, tool access, memory, and evaluations.
- Orchestration: AWS Lambda functions triggered by S3 events and workflow steps.
- Optional but strongly recommended: API Gateway or ALB for public APIs; EventBridge for reminders/schedules; Cognito for authentication if you want a faster managed auth path.

Reference flow

Capture -> Upload -> Store original -> OCR -> Normalize metadata -> Classify -> Extract due dates/actions -> Summarize -> Embed/index -> User search/Q&A -> Reminder events

1) User scans a document in the web app. The browser performs image cleanup (crop, perspective correction preview, compression) and uploads the image/PDF to S3 using a signed URL.

2) The S3 upload triggers Lambda. The function creates a Mail record in DynamoDB with status=uploaded and sends the file to Textract.

3) Textract returns raw OCR text and structure. A normalization Lambda converts this into canonical JSON: sender, recipient, dates, amounts, account numbers (masked), page count, and block-level text.

4) A classification Lambda assigns a top category and confidence score. The result is written to DynamoDB and attached as metadata to the document record.

5) A summarization Lambda or Bedrock workflow asks an LLM to produce: document type, key dates, payment amount, required actions, urgency, and suggested reminder date.

6) The normalized text and summary are indexed in Bedrock Knowledge Bases for retrieval-based search and question answering.

7) Reminder-worthy items produce an event or scheduled job for future notifications.

Strengths

- Fastest route to production because OCR, storage durability, security controls, and core AI infrastructure are managed.
- Best fit for documents with tables, forms, and semi-structured mail where Textract adds layout awareness beyond plain OCR.
- Operationally simpler for a small team: fewer custom services to build, patch, scale, and monitor.

Weaknesses

- Higher variable cost as document volume and model usage grow, especially if every scan triggers OCR, summarization, embedding, and multiple question-answering calls.
- Potential lock-in to AWS service contracts, data schemas, and retrieval patterns unless the application defines clean provider interfaces.
- Lambda/event-driven pipelines can introduce variable latency and more complex debugging than a single synchronous service.

3. Open-source / hybrid reference architecture

Recommended components

- Frontend/Web: same browser-based scanner experience as the AWS option.
- Storage: S3-compatible object storage. This can be Amazon S3 initially, or MinIO/Cloudflare R2/other compatible storage later.
- OCR: Tesseract, ideally wrapped with image preprocessing (deskew, denoise, thresholding, contrast) using Python/OpenCV.
- Application layer: Python services for ingestion, parsing, classification, summarization orchestration, and API delivery.
- Retrieval store: ChromaDB for embeddings and metadata retrieval.
- LLM orchestration: LangChain or a thinner custom abstraction layer.
- LLM inference: OpenAI or Anthropic APIs initially; self-hosted models can be added later if quality/cost justify it.
- Database: add Postgres or another transactional store for users, records, reminders, and audit logs. Chroma should not be the system-of-record database.

Reference flow

Capture -> Upload -> Python preprocessing -> Tesseract OCR -> Rule/ML/LLM classification -> Summary extraction -> Chroma indexing -> Search/Q&A API -> Reminder engine

The web app uploads a scan to object storage, then a Python ingestion service performs image preprocessing before sending the document through Tesseract.

The service stores OCR text, derived metadata, and an immutable pointer to the original artifact.

A classifier service uses rules, embeddings, and LLM prompts to assign categories and extract important fields.

Embeddings plus metadata are written to ChromaDB for search and retrieval. Transactional app data should go into a separate operational database.

LangChain can orchestrate retrieval and prompts, but the application should avoid deep framework coupling so future swaps remain easy.

Strengths

- Lowest long-run infrastructure cost potential if document volume is high and the team can manage the platform efficiently.
- Maximum portability across cloud providers and model vendors.
- More control over pipeline details, custom preprocessing, and optimization of specific document classes.

Weaknesses

- More engineering effort up front: image cleanup, OCR quality tuning, retries, deployment, scaling, and observability become your responsibility.
- Tesseract usually needs more preprocessing and quality tuning than managed document-analysis services for messy phone captures, forms, tables, and low-light scans.
- You will likely still pay external LLM inference costs if OpenAI or Anthropic remains part of the stack, so the architecture is not fully open-source in operating cost.

Important adjustment to the proposed open-source stack

Use a transactional database such as Postgres in addition to Chroma. Chroma is useful for retrieval, but the product still needs a durable source of truth for users, mail records, reminder state, category corrections, and audit history.

4. Architecture comparison and trade-offs

Dimension	AWS-managed	Open-source / hybrid	Winner
Time to MVP	Fastest. Most components are managed and integrate naturally through events.	Slower. Requires platform code for OCR tuning, indexing, and operations.	AWS
OCR quality on messy mail	Generally stronger for forms/tables and low-structure mail with less custom tuning.	Can be strong on clean scans, but quality drops sooner without preprocessing and tuning.	AWS
Unit cost at low volume	Acceptable, and usually justified by speed.	Potentially slightly cheaper infra, but engineering cost can dominate early.	AWS
Unit cost at high volume	Can become expensive because multiple managed services scale linearly with usage.	Can be cheaper if OCR and retrieval are optimized and self-hosted selectively.	Open
Operational burden	Lower. Managed security, scaling, and durability.	Higher. Team owns deployments, tuning, patching, and incident response.	AWS
Vendor lock-in	Higher without abstraction.	Lower by design.	Open
Flexibility	Good if interfaces are abstracted; otherwise moderate.	Highest. Swap components and providers as needed.	Open
Latency	Good for asynchronous ingestion; user-facing queries are fine with tuned retrieval.	Can be excellent if kept simple, but may degrade if too many custom stages are chained.	Tie
Compliance / audit posture	Easier to assemble with AWS-native controls and logging.	Possible, but requires more engineering discipline and tooling.	AWS
Best fit	Small team, fast MVP, production readiness.	Cost-optimized or control-heavy roadmap with strong platform capacity.	Depends

Recommendation

Launch the MVP on the AWS-managed architecture, but enforce provider abstraction in three places: OCR provider, LLM provider, and retrieval provider.

That gives the product the fastest delivery path today while preserving an escape hatch for cost optimization later.

If cost becomes the dominant concern after product-market validation, migrate selectively rather than all at once: first retrieval, then parts of classification, then OCR only if the quality delta is acceptable.

5. Website document-scanning implementation

The website scanner should be implemented as a browser-first capture flow rather than requiring users to email documents to themselves. On mobile web, use the device camera through browser media APIs; on desktop, support upload from phone photos, PDFs, or drag-and-drop.

Recommended client steps: live camera preview, auto edge detection, perspective correction preview, crop adjustment, grayscale/enhancement options, glare warning, blur detection, and multi-page capture.

Recommended server steps: create upload session -> signed URL upload -> virus/file validation -> store original -> enqueue OCR -> poll or push processing status -> render categorized result and summary.

#	Stage	What happens	Output
1	Capture	User captures 1..N pages or uploads a PDF/image.	Local preview
2	Client cleanup	Edge detect, crop, deskew preview, compress, and package pages.	Clean image bundle
3	Upload	Upload original and client-cleaned version to storage.	Stored object(s)
4	OCR	Run Textract or preprocessing+Tesseract.	Raw extracted text
5	Normalize	Parse dates, amounts, sender, account refs, page count, and layout clues.	Canonical JSON
6	Classify	Assign category + confidence + rationale.	Category record
7	Summarize	Generate actions, due dates, urgency, and answer seeds.	AI summary
8	Index	Write searchable metadata and embeddings.	Searchable mail item

Implementation note

Keep both the original file and the cleaned derivative. Originals are important for auditability and for future OCR reprocessing when better models become available.

6. Document and email categorization design

Target categories

Bills, bank, insurance, medical, government, personal, legal, and fallback categories such as other/unknown.

Recommended classification method

Use a hybrid hierarchical classifier, not a single black-box prompt.

Stage A - deterministic signals: sender/domain, mailbox routing rules, regex patterns, visible keywords, payment language, account-number patterns, and document-layout cues.

Stage B - ML/semantic scoring: embedding similarity or lightweight classifier over OCR text and subject/body text.

Stage C - LLM verification: an LLM receives the candidate classes, short evidence snippets, and a strict taxonomy prompt; it returns top class, alternate class, confidence, and extracted rationale.

Stage D - confidence policy: if confidence is below threshold, mark as needs-review or other. User corrections feed a labeled dataset for later fine-tuning.

Why this method is best

Method	Pros	Cons
Rules only	Cheap, explainable, fast.	Brittle; poor generalization to new senders and layouts.
Single LLM prompt only	Easy to start; flexible.	Higher cost, variable consistency, weaker auditability.
Custom document classifier only	Can become accurate with labels.	Needs dataset, training loop, and lifecycle management.
Hybrid rules + semantic model + LLM	Best balance of precision, robustness, explainability, and cold-start practicality.	More pipeline design work than a single method.

Email categorization specifics

Email messages should be classified from both metadata and content: sender address/domain, subject line, body text, attachments, and extracted text from PDFs/images when present.

Example evidence by class: bills -> due date + amount due + payment portal language; bank -> statement wording, account-ending patterns, transaction summaries; insurance -> policy, claim, premium; medical -> provider names, EOB, patient, balance; government -> agency names, official notices, tax language; legal -> contract, notice, court, counsel, agreement; personal -> person-to-person language and lack of institutional markers.

The email classifier and the scanned-document classifier should share the same taxonomy and confidence policy, but not necessarily the same feature weights.

Is this 'the Google method'?

No. The recommended method is product-driven, not vendor-driven. Google Cloud Document AI does offer custom classifier and custom splitter products, which are valid options when you want Google-managed document classification. But for MailVault, the better design principle is a vendor-neutral classification service with pluggable backends. That keeps the product free to use AWS-native, open-source, or Google-backed classifiers later without changing the user experience.

7. Suggested service boundaries

Service	Responsibility	Swap risk if abstracted well
Ingestion service	Upload sessions, file validation, state transitions.	Low
OCR provider	Textract or Tesseract wrapper.	Low
Classifier service	Rules, semantic scorer, LLM verification, confidence policy.	Low
Summary/Q&A service	Prompt templates, retrieval, answer formatting.	Low
Index service	Embeddings, metadata mapping, retrieval filters.	Medium
Reminder service	Due-date extraction, schedules, notifications.	Low

8. Final recommendation

Choose AWS-managed for MVP delivery: S3 + Textract + DynamoDB + Lambda + Bedrock Knowledge Bases, with Bedrock or another LLM provider behind an internal AI gateway.

Use a hybrid classifier service for both scanned documents and emails. Start with rules plus LLM verification, then add a trained classifier when corrected labels accumulate.

Implement the website scanning flow as an asynchronous pipeline with visible processing states. Users should never wait on a blocking synchronous OCR request.

Keep the system portable by defining provider interfaces from day one. That preserves the option to move specific stages to open-source components after the product proves demand and the cost profile becomes clear.

Source basis: MailVault PRD supplied by the requester; architecture choices and trade-offs based on the proposed stacks and current vendor documentation.